

Organização da Área de Trabalho por Tipo de Arquivo

Relatório da atividade · Módulo Introdução ao Claude Cowork · 23 de setembro de 2026

Item	Descrição
Atividade	Simulação da organização automática de uma Área de Trabalho, separando arquivos em pastas por tipo
Curso / módulo	Formação Claude Code 2026 — Introdução ao Claude Cowork
Ferramentas	Claude (sessão de trabalho em nuvem), Python 3, script <code>organizar_area_de_trabalho.py</code>
Ambiente	Pasta fictícia criada no ambiente de nuvem; nenhum arquivo real do computador foi alterado
Resultado	28 itens soltos reduzidos a 10 pastas organizadas + 2 itens mantidos na raiz

1. Resumo executivo

Foi pedida uma simulação de organização da Área de Trabalho, separando os arquivos de acordo com o seu tipo. Para isso, criou-se uma Área de Trabalho fictícia com 28 arquivos típicos de uso diário (fotos, documentos, planilhas, vídeos, instaladores, atalhos etc.) e desenvolveu-se um script em Python que identifica o tipo de cada arquivo pela extensão e o move para uma subpasta correspondente.

Resultado: 26 arquivos foram distribuídos em 10 pastas temáticas. Os 2 itens de sistema (um atalho do navegador e o arquivo *desktop.ini*) foram mantidos no lugar, de propósito. Nenhum arquivo foi sobrescrito ou perdido.

2. Objetivo

- Demonstrar como o Claude pode automatizar uma tarefa de organização de arquivos do dia a dia.
- Definir regras claras de classificação por tipo de arquivo.
- Validar o comportamento do processo em um ambiente seguro antes de aplicá-lo a arquivos reais.
- Entregar uma ferramenta reutilizável (o script) que pode ser aplicada depois em qualquer pasta.

3. Como a atividade foi realizada

Etapa	O que foi feito
1. Planejamento	Definição das categorias de arquivo e das extensões que pertencem a cada uma.
2. Desenvolvimento	Criação do script <code>organizar_area_de_trabalho.py</code> , com modo de simulação (padrão) e modo de execução (<code>--executar</code>).
3. Cenário de teste	Criação da pasta <code>Area_de_Trabalho_simulada</code> com 28 arquivos vazios que imitam uma Área de Trabalho real e bagunçada, incluindo casos difíceis (duplicado, extensão desconhecida, arquivo sem extensão, atalho).
4. Execução	Execução do script sobre a pasta simulada, gerando um relatório de cada arquivo movido.
5. Verificação	Conferência da estrutura final: 10 pastas criadas e 2 itens de sistema preservados na raiz.
6. Documentação	Elaboração deste relatório em PDF e de uma página web com o mesmo conteúdo.

4. Regras de classificação

O tipo de cada arquivo é identificado pela sua **extensão** (o final do nome, como `.pdf` ou `.jpg`), sem diferenciar maiúsculas de minúsculas. A tabela abaixo mostra para qual pasta cada extensão é enviada.

Pasta de destino	Extensões reconhecidas
Imagens	<code>.jpg .jpeg .png .gif .bmp .webp .svg .heic .ico</code>
Documentos	<code>.pdf .doc .docx .odt .txt .rtf .md</code>
Planilhas	<code>.xls .xlsx .ods .csv</code>
Apresentacoes	<code>.ppt .pptx .odp .key</code>
Videos	<code>.mp4 .mkv .avi .mov .wmv .webm</code>
Musicas	<code>.mp3 .wav .flac .aac .ogg .m4a</code>
Compactados	<code>.zip .rar .7z .tar .gz</code>
Instaladores	<code>.exe .msi .dmg .pkg .deb .apk</code>
Codigo	<code>.py .js .html .css .json .java .c .cpp .sql</code>
Outros	Qualquer extensão não listada acima, ou arquivo sem extensão
(permanece no lugar)	<code>.lnk .url .ini .desktop</code> — atalhos e arquivos de configuração do sistema

5. Cenário simulado (antes)

A Área de Trabalho fictícia começou com 28 arquivos soltos, todos na mesma pasta:

foto_praia_floripa.jpg	gravacao_aula_modulo3.mkv
print_tela_2026-09-20.png	podcast_ep12.mp3
logo_empresa.svg	audio_whatsapp.ogg
meme.gif	backup_fotos.zip
curriculo_atualizado.pdf	drivers.rar
curriculo_atualizado (copia).pdf	setup_vscode.exe
contrato_aluguel.docx	instalador_zoom.msi
anotacoes_reuniao.txt	script_teste.py
boleto_setembro.pdf	index.html
ideias.md	config.json
orcamento_ferias.xlsx	arquivo_estranho.xyz
gastos_mensais.csv	sem_extensao
apresentacao_projeto.pptx	Google Chrome.Ink
video_aniversario.mp4	desktop.ini

6. Resultado (depois)

Após a execução, cada arquivo foi para a pasta do seu tipo:

Pasta	Qtd.	Arquivos movidos
Documentos	6	anotacoes_reuniao.txt, boleto_setembro.pdf, contrato_aluguel.docx, curriculo_atualizado.pdf, curriculo_atualizado (copia).pdf, ideias.md
Imagens	4	foto_praia_floripa.jpg, logo_empresa.svg, meme.gif, print_tela_2026-09-20.png
Codigo	3	config.json, index.html, script_teste.py
Compactados	2	backup_fotos.zip, drivers.rar
Instaladores	2	instalador_zoom.msi, setup_vscode.exe
Musicas	2	audio_whatsapp.ogg, podcast_ep12.mp3
Outros	2	arquivo_estranho.xyz, sem_extensao
Planilhas	2	gastos_mensais.csv, orcamento_ferias.xlsx
Videos	2	gravacao_aula_modulo3.mkv, video_aniversario.mp4
Apresentacoes	1	apresentacao_projeto.pptx
Total	26	10 pastas criadas

Mantidos na raiz: *Google Chrome.Ink* e *desktop.ini*. Atalhos precisam continuar visíveis na Área de Trabalho, e o *desktop.ini* é um arquivo de configuração do sistema.

Estrutura final da Área de Trabalho

```
Area_de_Trabalho_simulada/
├── Apresentacoes/ (1)
├──Codigo/ (3)
├── Compactados/ (2)
├── Documentos/ (6)
├── Imagens/ (4)
├── Instaladores/ (2)
├── Musicas/ (2)
├── Outros/ (2)
├── Planilhas/ (2)
├── Videos/ (2)
├── Google Chrome.Lnk
└── desktop.ini
```

Indicadores

Indicador	Antes	Depois
Itens soltos na raiz	28	2
Pastas temáticas	0	10
Arquivos perdidos ou sobrescritos	—	0
Redução de itens visíveis na raiz	—	≈ 57% (de 28 para 12 itens: 10 pastas + 2 arquivos)

7. Cuidados de segurança incluídos no processo

- **Simulação por padrão:** sem o parâmetro `--executar`, o script apenas lista o que faria, sem mover nada. Isso permite revisar o plano antes de agir.
- **Sem sobrescrita:** se já existir um arquivo com o mesmo nome na pasta de destino, o novo recebe um sufixo, por exemplo *arquivo (1).pdf*.
- **Nada é apagado:** o script apenas move arquivos; não exclui nenhum.
- **Atalhos e arquivos de sistema preservados:** arquivos `.lnk`, `.url`, `.ini` e `.desktop` ficam onde estão.
- **Arquivos ocultos e pastas ignorados:** itens que começam com ponto e subpastas já existentes não são tocados.
- **Ambiente isolado:** nesta atividade, tudo aconteceu em uma pasta fictícia, sem risco para arquivos reais.

8. Como aplicar na Área de Trabalho real

O script pode ser usado em qualquer pasta. No Linux, a Área de Trabalho normalmente fica em `~/Área de Trabalho`. O processo recomendado tem dois passos:

```
# 1) Ver o que seria feito (não move nada)
python3 organizar_area_de_trabalho.py "~/Área de Trabalho"

# 2) Se estiver tudo certo, executar de verdade
python3 organizar_area_de_trabalho.py "~/Área de Trabalho" --executar
```

Outra opção é pedir ao próprio Claude, com o computador vinculado à sessão: ele solicita acesso à pasta, mostra a lista do que seria movido e só organiza depois da sua aprovação.

9. Conclusão e próximos passos

A simulação mostrou que a organização por tipo de arquivo pode ser automatizada de forma simples, previsível e segura. As regras cobrem os formatos mais comuns, e os casos especiais (duplicados, extensões desconhecidas, atalhos) foram tratados sem perda de dados.

- Aplicar o script, em modo simulação, à Área de Trabalho real e revisar a lista gerada.
- Ajustar as categorias conforme necessário, por exemplo criando uma pasta separada para PDFs ou para prints de tela.
- Usar o mesmo processo em outras pastas acumulativas, como Downloads.
- Opcionalmente, agendar a organização para rodar periodicamente.

Anexo A — Código-fonte do script

Arquivo *organizar_area_de_trabalho.py* (Python 3.10 ou superior):

```
"""
Organiza uma pasta (ex.: Área de Trabalho) separando os arquivos em subpastas por tipo.

Uso:
python organizar_area_de_trabalho.py CAMINHO_DA_PASTA          # simulação (não move nada)
python organizar_area_de_trabalho.py CAMINHO_DA_PASTA --executar # move de verdade
"""

import sys
import shutil
from pathlib import Path
from collections import defaultdict

CATEGORIAS = {
    "Imagens": {" .jpg", ".jpeg", ".png", ".gif", ".bmp", ".webp", ".svg", ".heic", ".ico"},
    "Documentos": {" .pdf", ".doc", ".docx", ".odt", ".txt", ".rtf", ".md"},
    "Planilhas": {" .xls", ".xlsx", ".ods", ".csv"},
    "Apresentacoes": {" .ppt", ".pptx", ".odp", ".key"},
    "Videos": {" .mp4", ".mkv", ".avi", ".mov", ".wmv", ".webm"},
    "Musicas": {" .mp3", ".wav", ".flac", ".aac", ".ogg", ".m4a"},
    "Compactados": {" .zip", ".rar", ".7z", ".tar", ".gz"},
    "Instaladores": {" .exe", ".msi", ".dmg", ".pkg", ".deb", ".apk"},
    "Codigo": {" .py", ".js", ".html", ".css", ".json", ".java", ".c", ".cpp", ".sql"},
}

IGNORAR = {" .lnk", ".url", ".ini", ".desktop"} # atalhos e arquivos de sistema ficam onde estão

def categoria_de(arquivo: Path) -> str | None:
    ext = arquivo.suffix.lower()
    if ext in IGNORAR:
        return None
    for nome, extensoes in CATEGORIAS.items():
        if ext in extensoes:
            return nome
    return "Outros"

def destino_livre(destino: Path) -> Path:
    """Evita sobrescrever: arquivo.pdf -> arquivo (1).pdf"""
    if not destino.exists():
        return destino
    n = 1
    while True:
        candidato = destino.with_name(f"{destino.stem} ({n}){destino.suffix}")
        if not candidato.exists():
            return candidato
        n += 1

def organizar(pasta: Path, executar: bool) -> None:
    plano = defaultdict(list)
    ignorados = []
    for item in sorted(pasta.iterdir()):
        if not item.is_file() or item.name.startswith("."):
            continue
        cat = categoria_de(item)
        if cat is None:
            ignorados.append(item.name)
        else:
            plano[cat].append(item)

    modo = "EXECUTANDO" if executar else "SIMULAÇÃO (nada será movido)"
    print(f"== {modo} em: {pasta} ==\n")
    total = 0
    for cat in sorted(plano):
        print(f"{cat}/ ({len(plano[cat])})")
        for arq in plano[cat]:
            print(f"    ← {arq.name}")
            if executar:
                (pasta / cat).mkdir(exist_ok=True)
```

```

        shutil.move(str(arq), destino_livre(pasta / cat / arq.name))
        total += 1
    if ignorados:
        print(f"\nMantidos no lugar (atalhos/sistema): {' '.join(ignorados)}")
    print(f"\nTotal: {total} arquivos {'movidos' if executar else 'seriam movidos'} "
          f"para {len(plano)} pastas.")

if __name__ == "__main__":
    if len(sys.argv) < 2:
        print(__doc__)
        sys.exit(1)
    organizar(Path(sys.argv[1]).expanduser(), executar="--executar" in sys.argv)

```